Experiential Learning
Project Assessment-I
on
“Mad Libs Generator
Python Programming Language (N-PCCCM304P)

Presented By

Saloni Manoj Edalwar
(CM25049)



Under the Guidance
of
Mrs. Mayuri Getme

Session: 2026-27 (ODD)

Department of CSE (AI&ML)

S. B. JAIN INSTITUTE OF TECHNOLOGY MANAGEMENT AND RESEARCH, NAGPUR
An Autonomous Institute, Affiliated to RTMNU, Nagpur

Problem Statement & Objectives

Problem Statement

Develop a basic Python-based Mad Libs Generator that collects different words such as nouns, verbs, adjectives, and places from the user. The program should use these inputs with predefined story templates to generate a creative and entertaining story..

Objectives

1. To develop a simple Mad Libs Generator using Python.
2. To collect different types of words from the user.
3. To generate creative stories using predefined templates
4. To demonstrate basic Python concepts like input, variables, strings, and conditional statements.
5. To make the program interactive and entertaining

Algorithm

01 **Start** Begin the Mad Libs Generator program.

02 **Choose Template** Display the menu and let the user select a story template.

03 **Take Inputs** Ask for words such as name, noun, verb, adjective, adverb and place.

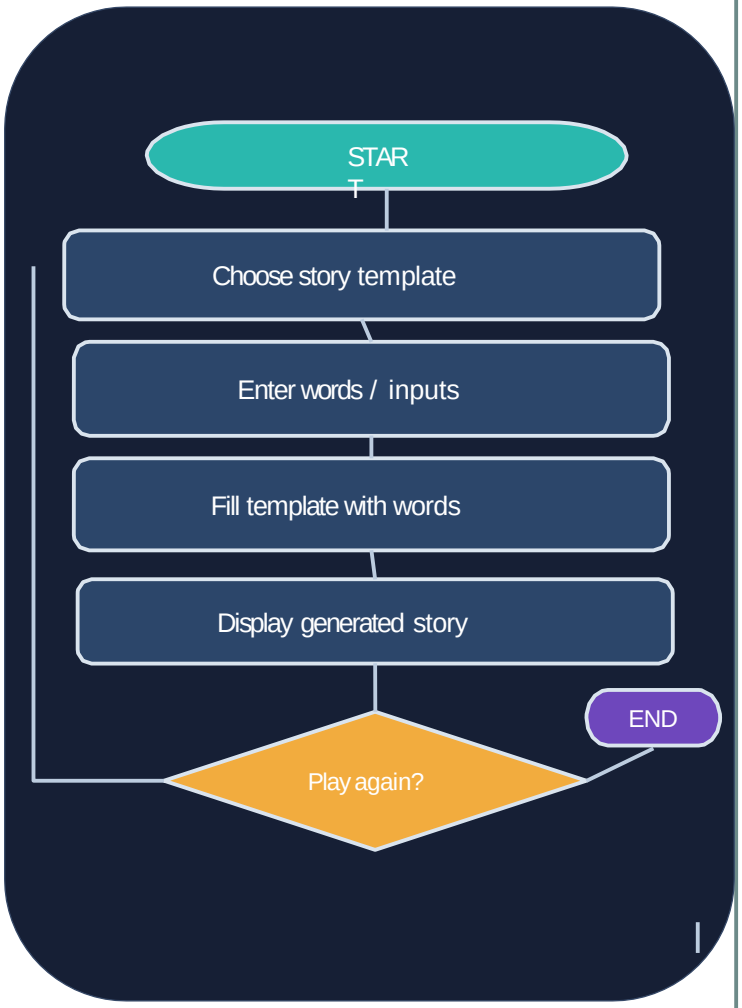
04 **Build Story** Insert the entered words into the selected predefined template.

05 **Generate** Create the complete and entertaining story.

06 **Display & Stop** Show the generated story and end the program.

Pseudocode & Flowchart

```
START
Display available story templates
Ask user to choose a template
Collect required words:
  noun, verb, adjective, adverb, place…
Insert words into the selected template
Generate the completed story
Display the funny story
Ask: "Play again?"
IF Yes → repeat from
template selection
IF No  → END
```



Understanding of Python Logic

01

INPUT

Take words from the user

noun • verb • adjective • place

02

PROCESS

Store and organize the inputs

variables • lists • strings

03

LOGIC

Insert words into a template

conditions • loops • functions

04

OUTPUT

Generate and display the story

formatted text • user feedback

PYTHON LOGIC IN THE PROJECT

Variables	Store each word entered by the user
String Formatting	Combine words with the story template
If / Else	Validate choices and handle play-again logic
Loops	Allow multiple rounds of story generation
Functions	Separate input, story generation and display

KEY IDEA

Input → Process → Decision → Output

GitHub Workflow

- Created a GitHub account and repository: Mad libs generator in python
- Added a README.md explaining the project overview and usage
- Maintained commit history to reflect individual contribution
- Used .gitignore to exclude unnecessary files

Sample Commit Messages

- "Initial commit: Add Mad Libs Generator"
- "Add README documentation for Mad Libs project"
- "Add Mad Libs story generator program"
- "Add Mad Libs story generator program"
- "Add pseudocode and flowchart documentation"
- "Add Mad Libs Generator with documentation"

Phase 2: What I Added

PHASE 2

From a basic story generator to a more interactive program

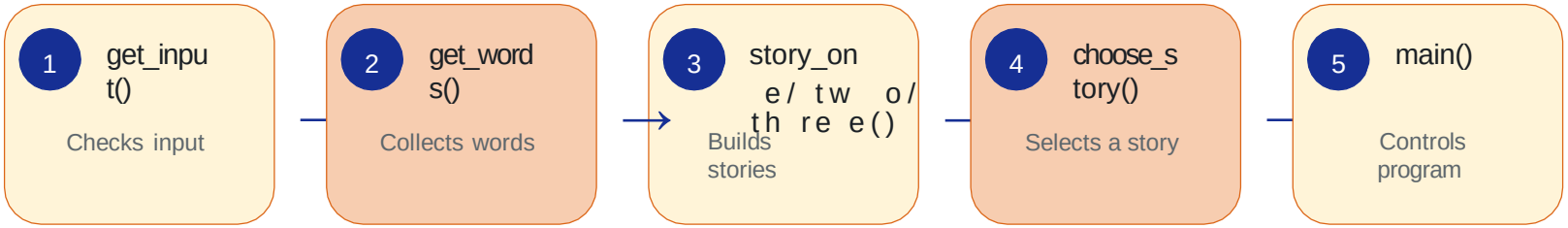
70% PROJECT DEVELOPMENT

1. Multiple stories — added 3 different story templates.
2. Random choice — the program can select a story automatically.
3. Input checking — blank input is not accepted.
4. Menu system — Random Story, Choose Story, or Exit.
5. Functions — each task is kept in a simple function.
6. Repeat use — more than one story can be generated in one run.

Code Structure

PHASE 2

Simple functions make the program easy to understand and explain



Example: collecting user input

```
def
get_input(message):
    while True:
        value = input(message).strip()
        if value != "":
            return value
        print("Please enter
        something!")

def get_words():
    words = {}
    words["name"] = get_input("Enter a name: ")
    words["noun"] = get_input("Enter a noun: ")
```

Coding: Story Templates & Random Selection

PHASE 2

1. Story template

```
def story_one(words):
    return f"""
    Once upon a time,
{words["name"]}
    found a {words["animal"]}.
    The day became very
{words["adjective"]}!
    """
```

2. Random selection

```
def generate_random_story(words):
    stories = [
        story_one,
        story_two,
        story_three
    ]
    return random.choice(stories)
(words)
```

Why this is useful

- More variety a random story
- Less repetition get
- User can choose or

Coding: Menu & Input Validation

PHASE 2

These parts make the program interactive and easier to use

MENU

```
1.   Generate a
Random Story
2. Choose a Story
3. Exit
```

VALIDATION

```
value = input(message).strip()

if value != "":
    return value
else:
    print("Please enter
something!")
```

Easy explanation

The menu tells the user what they can do. If the user leaves a word blank, the program asks again instead of accepting empty data.

Execution 1: Choosing a Story

PHASE 2

Sample run of the Phase 2 program

```
MAD LIBS STORY GENERATOR
1. Generate a Random Story
2. Choose a Story
3. Exit
Enter your choice: 2

Enter the required words:
name: Saloni   adjective: creative   noun: book
verb: explore   adverb: quickly   place: Nagpur
animal: cat    food: pizza    emotion: happy    number: 3

Select a Story:
1. The Funny Adventure
2. The Great Adventure
3. A Very Strange Day
Enter your choice (1-3): 2
```

Execution 2: Generated Story

PHASE 2

The program combines the inputs into a complete story

```
YOUR STORY

THE GREAT ADVENTURE
One morning, Saloni woke up feeling very happy.
Saloni packed a book and started to explore
towards Nagpur. On the way, a creative cat appeared.
The animal shouted, "Give me some pizza!"
Saloni was surprised and ran quickly.
After running for 3 minutes, Saloni finally
reached Nagpur.
It was an adventure that Saloni would never forget!

Program can now return to the main menu.
```

Phase 3: Final Enhancement

PHASE 3

Moving from a working generator to a polished, user-friendly project

100% PROJECT DEVELOPMENT

1. **Improved interface**
   A clear title, organized menu, and readable story output.

2. **More story choices**
   Basic story, random story generation, and manual story selection.

3. **Safer input handling**
   Prevents blank words and checks that the number is a positive integer.

4. **Project information**
   An About section explains the project and Python concepts used.

Phase 3: Enhanced Features

PHASE 3

New additions that improve usability and presentation

01

Colourful terminal

Uses terminal colours to highlight headings, messages, and story output.

02

Three story themes

School day, magical adventure, and a funny restaurant.

03

Repeat without restarting

Returns to the main menu after each action, so users can keep playing.

04

About this project

Shows the language, project phases, and key Python concepts.

Phase 3: Input Validation

PHASE 3

The program guides the user when input is missing or incorrect

VALIDATION EXAMPLES

```
def get_input(prompt):

    while True:

        answer = input(prompt).strip()

        if answer:

            return answer

        print("Input cannot be empty")


number = int(answer)

if number > 0:

    return number
```

WHY IT MATTERS

1. **No blank words**
   The program asks again if a required word is empty.

2. **Valid number**
   Only a positive whole number is accepted.

3. **Clear feedback**
   Helpful messages tell the user what to correct.

Phase 3: Testing & Sample Run

PHASE 3

Test different choices to check that the program behaves correctly

TEST CASE	INPUT	EXPECTED RESULT
Menu choice	1, 2, or 3	Creates the selected type of story
Empty word	Press Enter	Shows a message and asks again
Invalid menu	9	Shows an invalid-choice message
Invalid number	abc or 0	Asks for a valid positive number
Exit	5	Displays goodbye message and stops

Testing helps identify mistakes and makes the program more reliable.

Phase 3: Final Outcome & Future Scope

PHASE 3

Summary of the completed Mad Libs Generator project

PROJECT OUTCOME

- ✓ Interactive Python menu
- ✓ Four ways to explore the program
- ✓ Three creative story templates
- ✓ Input checking and clear feedback
- ✓ Reusable functions and formatted history

IMPROVEMENTS

- Save generated stories to a text file
- Add more story templates and them
- Build a graphical user interface (GU
- add user names and story

Conclusion: Python functions, strings, loops, conditions, and random selection can create a simple interactive storytelling application.

Thank You